

```
1 === "1" // false
true === "true" // false
```

```
var x = 1;
var y = 2;
if (x = y) {
  console.log(x);
}
// "2"
```

```
if (x = 2) { // 不报错
if (2 = x) { // 报错
```

小技巧

```
for (var i = 0; i < 5; i++) {
  console.log(i);
  if (i === 3)
    break;
}
// 0
// 1
// 2
// 3
```

```
top:
for (var i = 0; i < 3; i++){
  for (var j = 0; j < 3; j++){
    if (i === 1 && j === 1) break top;
    console.log('i=' + i + ', j=' + j);
  }
}
// i=0, j=0
// i=0, j=1
// i=0, j=2
// i=1, j=0
```

双重循环区块，**break**命令后面加上了**top**标签（注意，**top**不用加引号），满足条件时，直接跳出双层循环。如果**break**语句后面不使用标签，则只能跳出内层循环，进入下一次的外层循环

```
foo: {
  console.log(1);
  break foo;
  console.log('本行不会输出');
}
console.log(2);
// 1
// 2
```

标签也可以用于跳出代码块
示例中代码执行到break foo，就会跳出区块

```
top:
  for (var i = 0; i < 3; i++){
    for (var j = 0; j < 3; j++){
      if (i === 1 && j === 1) continue top;
      console.log('i=' + i + ', j=' + j);
    }
  }
// i=0, j=0
// i=0, j=1
// i=0, j=2
// i=1, j=0
// i=2, j=0
// i=2, j=1
// i=2, j=2
```

continue语句也可以与标签配合使用
示例中continue命令后面有一个标签名，满足条件时，
会跳过当前循环，直接进入下一轮外层循环
如果continue语句后面不使用标签，则只能进入下一轮的
内层循环，去掉top试一下！（i=1,j=2）

```
typeof 123 // "number"
typeof '123' // "string"
typeof false // "boolean"
```

数值、字符串、布尔值分别返回
number、string、boolean

```
function f() {}
typeof f
// "function"
```

函数返回function，对象的子类型之一
(Brendan Eich觉得：函数太常用了，如果
typeof 返回 "object"，开发者会区分函数
和普通对象很困难。

```
typeof undefined
// "undefined"
```

undefined返回undefined

```
typeof window // "object"
typeof {} // "object"
typeof [] // "object"
```

对象返回object
在JavaScript内部，数组本质上
只是一种特殊的对象

```
typeof null // "object"
```

null返回object

```
typeof null // "object"
```

null返回object

```
if (!undefined) {  
  console.log('undefined is false');  
}  
// undefined is false  
  
if (!null) {  
  console.log('null is false');  
}  
// null is false  
  
undefined == null  
// true
```

```
Number(null) // 0  
5 + null // 5
```

```
Number(undefined) // NaN  
5 + undefined // NaN
```

```
1  typeof null      // "object" (因为一些以前的原因而不是'null')  
2  typeof undefined // "undefined"  
3  null === undefined // false  
4  null == undefined // true  
5  null === null // true  
6  null == null // true  
7  !null //true  
8  isNaN(1 + null) // false  
9  isNaN(1 + undefined) // true
```

undefined vs null in JavaScript
<https://howtodoinjava.com/typescript/undefined-vs-n>

```
var myUndefinedVar;      //undefined  
var myNullVar = null;    //null  
  
myUndefinedVar + 100;     //NaN  
  
myNullVar + 100;         //100
```

```
if (``) {  
  console.log('true');  
}  
// 没有任何输出
```

if命令后面的判断条件，预期应该是一个布尔值，所以JavaScript自动将空字符串，转为布尔值false，导致程序不会进入代码块，所以没有任何输出

```
if ([]) {  
  console.log('true');  
}  
// true  
  
if ({}) {  
  console.log('true');  
}  
// true
```

注意，空数组（[]）和空对象（{}）对应的布尔值，都是true

```
>> Boolean([])  
← true  
>> Boolean({})  
← true
```

```
123e3 // 123000  
123e-3 // 0.123  
-3.1E+12  
.1e-23
```

```
1234567890123456789012  
// 1.2345678901234568e+21  
  
123456789012345678901  
// 123456789012345680000
```

```
// 小数点后紧跟5个以上的零，  
// 就自动转为科学计数法  
0.0000003 // 3e-7  
  
// 否则，就保持原来的字面形式  
0.000003 // 0.000003
```

```
0xff // 255  
0o377 // 255  
0b11 // 3
```

```
-0 === +0 // true  
0 === -0 // true  
0 === +0 // true
```

```
+0 // 0  
-0 // 0  
(-0).toString() // '0'  
(+0).toString() // '0'
```

```
(1 / +0) === (1 / -0) // false
```

```
> 1/+0  
< Infinity  
> 1/-0  
< -Infinity
```

```
> 5 - 'x' // NaN
< NaN
> Math.sqrt(-1) // NaN
< NaN
> 0 / 0 // NaN
< NaN
> typeof NaN // 'number'
< "number"
> NaN === NaN // false
< false
> [NaN].indexOf(NaN) // -1
< -1
> Boolean(NaN) // false
< false
> NaN + 32 // NaN
< NaN
> NaN+NaN
< NaN
```

```
// 场景一
Math.pow(2, 1024)
// Infinity

// 场景二
0 / 0 // NaN
1 / 0 // Infinity
```

```
Infinity === -Infinity // false
```

```
1 / -0 // -Infinity
```

```
-1 / -0 // Infinity
```

```
Infinity > NaN // false
```

```
-Infinity > NaN // false
```

```
Infinity < NaN // false
```

```
-Infinity < NaN // false
```

```
parseInt('123') // 123
```

```
> parseInt(123, 20)
```

```
< 443
```

```
parseInt(' 81') // 81
```

如果字符串头部

```
parseInt(1.23) // 1
```

```
// 等同于
```

```
parseInt('1.23') // 1
```

如果parseInt的参数
为字符串再转换

```
parseInt('8a') // 8  
parseInt('12**') // 12  
parseInt('12.34') // 12  
parseInt('15e2') // 15  
parseInt('15px') // 15
```

```
parseInt('abc') // NaN  
parseInt('.3') // NaN  
parseInt('') // NaN  
parseInt('+') // NaN  
parseInt('+1') // 1
```

```
parseInt('0x10') // 16
```

```
parseFloat('3.14') // 3.14
```

```
parseFloat('314e-2') // 3.14  
parseFloat('0.0314E+2') // 3.14
```

```
parseFloat('3.14more non-digit characters') // 3.14
```



```
isNaN(NaN) // true
isNaN(123) // false
```

```
isNaN('Hello') // true
// 相当于
isNaN(Number('Hello')) // true
```

```
function myIsNaN(value) {
  return value !== value;
}
```

```
>> Number([])
```

```
< 0
```

```
>> Number([100])
```

```
< 100
```



```
> Number([100, 20, 30])
```

```
< NaN
```

```
isFinite(Infinity) // false
isFinite(-Infinity) // false
isFinite(NaN) // false
isFinite(undefined) // false
isFinite(null) // true
isFinite(-1) // true
```

```
var s = 'hello';  
s[0] // "h"  
s[1] // "e"  
s[4] // "o"
```

```
// 直接对字符串使用方括号运算符  
'hello'[1] // "e"
```

```
'abc'[3] // undefined  
'abc'[-1] // undefined  
'abc'['x'] // undefined
```

```
var s = 'hello';
```

```
delete s[0];  
s // "hello"
```

```
s[1] = 'a';  
s // "hello"
```

```
s[5] = '!';  
s // "hello"
```

```
var s = 'hello';  
s.length // 5
```

```
s.length = 3;  
s.length // 5
```

```
s.length = 7;  
s.length // 5
```

```
>> -1 % 2  
← -1  
>> 1 % 2  
← 1
```

```
> 1%-2  
← 1
```

```

>> +true
< 1
>> +[]
< 0
>> +{}
< NaN
>> -false
< -0
>> +false
< 0

```

空数组转换为字符串后是空字符串，再将空字符串转换为数字时会得到 0

空对象在转换为字符串时会变成 "[object Object]"

```

>> var x = 1
< undefined
>> -x
< -1
>> x
< 1

```

```

>> 2**4
< 16
>> 2**3**2 == 2**9
< true
>> (2**3)**2
< 64
>> 2**3**2
< 512

```

```

>> 'cat' > 'dog'
< false
>> 'cat' > 'catdog'
< false
>> 'cat' < 'Cat'
< false
>> '大' > '小'
< false

```

```

> 'cat'>'Cat'
< true
> '大'<'小'
< true

```

都是false

```
5 > '4' // true
// 等同于 5 > Number('4')
// 即 5 > 4

true > false // true
// 等同于 Number(true) > Number(false)
// 即 1 > 0

2 > true // true
// 等同于 2 > Number(true)
// 即 2 > 1
```

```
1 > NaN // false
1 <= NaN // false
'1' > NaN // false
'1' <= NaN // false
NaN > NaN // false
NaN <= NaN // false
```

JavaScript 的比较操作符会根据操作数的类型来自动处理，但开发人员应该明确操作数的类型，以避免产生预期之外的结果。

```
> 5 > 'cat'
< false
> 5 < 'cat'
< false
> 5 == 'cat'
< false
> '5' < 'cat'
< true
```

```
1 === "1" // false
true === "true" // false
```

```
1 === 0x1 // true
```

```
NaN === NaN // false
+0 === -0 // true
```

```
{ } === { } // false
[ ] === [ ] // false
(function () { }) === function () { } // false
```

```
var v1 = { };
var v2 = v1;
v1 === v2 // true
```

```

>> var o1 = {};
    var o2 = {};
< undefined
>> o1 > o2
< false
>> o1.valueOf().toString()
< "[object Object]"
>> o2.valueOf().toString()
< "[object Object]"
>> o1 === o2
< false
>> o1 == o2
< false

```

o1 和 o2 指向两

```

undefined === undefined // true
null === null // true

```

```

>> 1 == 1.0
< true
>> 1 === 1.0
< true
>> var x = [];
    var y = [];
< undefined
>> x == y
< false

```

```

1 !== '1' // true
// 等同于
!(1 === '1')

```

```

1 == true // true
// 等同于 1 === Number(true)

0 == false // true
// 等同于 0 === Number(false)

2 == true // false
// 等同于 2 === Number(true)

2 == false // false
// 等同于 2 === Number(false)

'true' == true // false
// 等同于 Number('true') === Number(true)
// 等同于 NaN === 1

```

```

'' == 0 // true
// 等同于 Number('') === 0
// 等同于 0 === 0

'' == false // true
// 等同于 Number('') === Number(false)
// 等同于 0 === 0

'1' == true // true
// 等同于 Number('1') === Number(true)
// 等同于 1 === 1

'\n 123 \t' == 123 // true
// 因为字符串转为数字时, 省略前置和后置的空格

```

```

// 对象与数值比较时, 对象转为数值
[1] == 1 // true
// 等同于 Number([1]) == 1

// 对象与字符串比较时, 对象转为字符串
[1] == '1' // true
// 等同于 String([1]) == '1'
[1, 2] == '1,2' // true
// 等同于 String([1, 2]) == '1,2'

// 对象与布尔值比较时, 两边都转为数值
[1] == true // true
// 等同于 Number([1]) == Number(true)
[2] == true // false
// 等同于 Number([2]) == Number(true)

```

```

false == null // false
false == undefined // false

0 == null // false
0 == undefined // false

undefined == null // true

```

```

> 0==Number(null)
< true

> 0==null
< false

```

```
0 == ''          // true
0 == '0'         // true

2 == true        // false
2 == false       // false

false == 'false' // false
false == '0'     // true

false == undefined // false
false == null      // false
null == undefined  // true

' \t\r\n ' == 0    // true  Number("") 的结果是 0 (制表符\t 回车\r 换行\n)
```

```
1 != '1' // false
```

// 等同于

```
!(1 == '1')
```

```
!undefined // true
```

```
!null // true
```

```
!0 // true
```

```
!NaN // true
```

```
!"" // true
```

```
!54 // false
```

```
!'hello' // false
```

```
![] // false
```

```
!{} // false
```



```
't' && '' // ""
't' && 'f' // "f"
't' && (1 + 2) // 3
'' && 'f' // ""
'' && '' // ""
```

第一个运算符的布尔值为true

第一个运算符的布尔值为false

```
var x = 1;
(1 - 1) && (x += 1) // 0
x // 1
```

第一个运算符的布尔值为false，不对第二个运算符求值
这种跳过第二个运算符的机制，被称为“短路”

```
>> true && 'foo' && NaN && 4 && 'foo' && true
< NaN
>> 1 && 2 && (7+8)
< 15
```

```
't' ? 'hello' : 'world' // "hello"
0 ? 'hello' : 'world' // "world"
```

```
// 数值：转换后还是原来的值
Number(324) // 324

// 字符串：如果可以解析为数值，则转换为相应的数值
Number('324') // 324

// 字符串：如果不可以被解析为数值，返回 NaN
Number('324abc') // NaN

// 空字符串转为0
Number('') // 0
```

```
// 布尔值：true 转成 1, false 转成 0
Number(true) // 1
Number(false) // 0

// undefined：转成 NaN
Number(undefined) // NaN

// null：转成0
Number(null) // 0
```

```
parseInt('42 cats') // 42
Number('42 cats') // NaN
```

Number函数将字符串转为数值，要比parseInt函数严格得多，parseInt逐个解析字符，而Number函数整体转换字符串的类型，只要有一个字符无法转成数值，整个字符串会被转为NaN


```
String(123) // "123"  
String('abc') // "abc"  
String(true) // "true"  
String(undefined) // "undefined"  
String(null) // "null"
```

```
Boolean(undefined) // false  
Boolean(null) // false  
Boolean(0) // false  
Boolean(NaN) // false  
Boolean('') // false
```

```
'5' + 1 // '51'  
'5' + true // "5true"  
'5' + false // "5false"  
'5' + {} // "5[object Object]"  
'5' + [] // "5"  
'5' + function (){} // "5function (){}"  
'5' + undefined // "5undefined"  
'5' + null // "5null"
```

```

'5' - '2' // 3
'5' * '2' // 10
true - 1 // 0
false - 1 // -1
'1' - 1 // 0
'5' * [] // 0
false / '5' // 0
'abc' - 1 // NaN
null + 1 // 1
undefined + 1 // NaN

```

```

+'abc' // NaN
-'abc' // NaN
+true // 1
-true // 0

```

```

> null + '1'
< 'null1'
> null + 1
< 1

```

null转为数值时为0，而undefined
 转为数值时为NaN

School of Cyber Security, BUPT #